

INGEGNERIA DEL SOFTWARE

L'ingegneria del software nasce quando, con la diffusione dei computer, si separano due figure che inizialmente coincidevano: l'utente e il programmatore.

Con il tempo i programmatori hanno iniziato a sviluppare software per altre persone facendo nascere così l'ingegneria del software, che si occupa non solo di scrivere codice, ma di organizzare e gestire lo sviluppo del software in modo sistematico e controllato.

Il Ruolo dell'Ingegnere del software

E' una figura che si occupa della progettazione e gestione di sistemi software complessi. Oltre a saper programmare, conoscere algoritmi, strutture dati e linguaggi di programmazione, DEVE essere in grado di lavorare su sistemi grandi e complessi ossia saper costruire modelli, ragionare a diversi modelli di astrazione, riutilizzare componenti già esistenti per risparmiare tempo e costi, lavorare in gruppo e gestire progetti.

Differenza tra:

- PROGRAMMAZIONE IN PICCOLO : scrivere codice cose legate più alle cose che consistono in ambito "programmatorio".
- PROGRAMMAZIONE IN GRANDE : saper progettare e gestire i sistemi software complessi

Risorse dell'Ingegnere del Software

Per sviluppare software complessi, l'ingegnere del software utilizza diversi modelli. Questi modelli servono per descrivere il prodotto software, il processo di sviluppo e le risorse necessarie. Servono per valutare e controllare la qualità del software.

Questi modelli si basano sull'esperienza e su dati empirici raccolti nel tempo.

Il ciclo di vita del software

Il software attraversa diverse fasi, che insieme formano il ciclo di vita del software.

Il modello tradizionale è il modello a cascata, in cui si passa da una fase all'altra in modo sequenziale.



1. cioè si definisce cosa deve fare il software
2. si definisce la struttura e l'architettura che comporranno il sistema
3. si scrive il codice e si testano i singoli moduli
4. si integrano tutti i moduli e si testa il sistema completo con il test di Integrazione e il test di Sistema
5. Infine il software viene consegnato e poi mantenuto nel tempo per correggere errori e adattarlo a nuove esigenze

LA QUALITÀ

La qualità non riguarda solo il prodotto finale, ma anche il processo con cui viene sviluppato. Infatti, la qualità del prodotto dipende dalla qualità del processo con cui è stato costruito.

Nell'ambito della qualità del software si sta assistendo nell'ultimo decennio ad un processo di convergenza virtuoso e progressivo tra:

-QUALITA' di PROCESSO

-QUALITA' di PRODOTTO

Il processo di produzione del software consiste essenzialmente nel progetto e nell'implementazione.

Questo processo deve soddisfare opportuni criteri che assicurino la produzione di software di elevata qualità

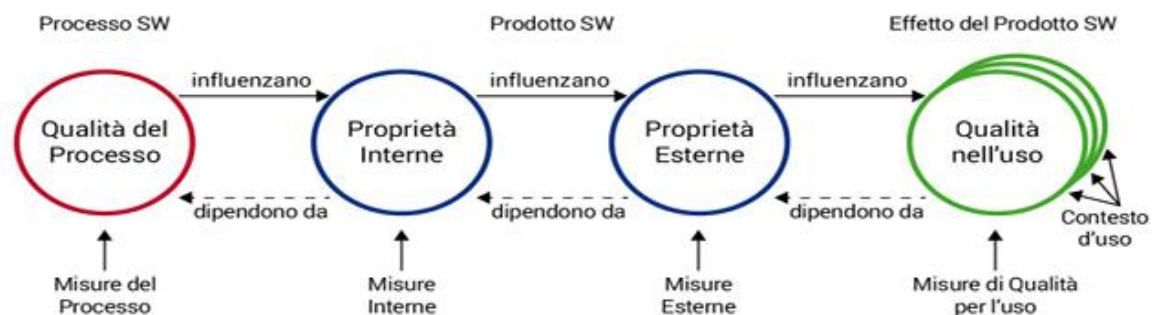
Le tre prospettive della qualità

- QUALITÀ IN USO (percepita): esprime l'efficacia ed efficienza con cui il software serve le esigenze dell'utente, ed è correlata alla percezione diretta dell'utente

- QUALITÀ INTERNA: esprime la misura in cui il codice software possiede una serie di attributi statici, indipendentemente dall'ambiente di utilizzo e dall'utente

- QUALITÀ ESTERNA: esprime il comportamento dinamico del software, nell'ambito d'uso

Queste tre qualità sono collegate tra loro: se il codice è scritto bene e il software funziona bene, allora anche l'utente percepirà un software di qualità.



Caratteristiche di qualità del software

- Correttezza: il software fa ciò che deve fare
- Affidabilità: funziona correttamente nel tempo
- Robustezza: funziona anche in situazioni impreviste
- Usabilità: facilità di utilizzo
- Manutenibilità: facilità di modifica e correzione
 - si identifica la causa di un malfunzionamento e si elimina [RIPARABILITÀ]
 - si aggiungono, sottraggono e si modificano capacità e funzioni del sistema [EVOLVIBILITÀ]
- Riutilizzabilità: possibilità di riutilizzare parti di software

Impatto economico della Qualità del Software

La qualità del software ha un forte impatto economico. Influisce sui costi di sviluppo, manutenzione e supporto, sui ricavi che il software può generare, sui costi operativi delle aziende che lo utilizzano e sulle opportunità di business. Un software di qualità quindi non è solo un fatto tecnico, ma anche economico.

I TIPI DI PRODOTTI SOFTWARE

Esistono tre tipi di prodotti software:

1- PROGRAMMA

È un insieme di istruzioni autoconsistenti rispetto ad uno o più obiettivi, un software scritto spesso per uso personale, con poca documentazione e vita breve.

2- APPLICAZIONE

È un insieme di programmi interagenti tra loro, con interfaccia e documentazione, destinato agli utenti, hanno bassi livelli di qualità.

3- SISTEMA SOFTWARE

È un prodotto complesso, con alta qualità, documentazione completa, lunga vita e destinato a molti utenti. L'usabilità e la portabilità sono caratteristiche chiave. Per poter essere redditizio, un sistema software deve avere una lunga vita e deve invecchiare lentamente

Applicazioni per l'Impresa

Le applicazioni per l'impresa sono sistemi software complessi che gestiscono grandi quantità di dati, accessi contemporanei di molti utenti e molte interfacce, caratterizzati da:

- Dati persistenti, necessari per passare le informazioni tra differenti applicazioni e tra differenti esecuzioni della stessa applicazione
- Rilevanti volumi di dati

- Accesso concorrente ai dati
- Rilevante numero di schermate componenti l'interfaccia
- Integrazione tra applicazioni che supportano domini differenti della stessa impresa

Sistemi in Tempo Reale

Devono rispondere a eventi esterni entro un tempo prestabilito, spesso molto breve. Sono spesso utilizzati in contesti critici, la caratteristica principale è la risposta.

Sono orientati al controllo con caratteristiche di affidabilità e salvaguardia/protezione.

Sistemi Embedded

I sistemi embedded sono sistemi software che fanno parte di altri sistemi più grandi e spesso non hanno un'interfaccia diretta con l'utente. Comunicano invece con altri dispositivi e componenti del sistema di cui fanno parte.

PROBLEMI DELLA INGEGNERIA DEL SOFTWARE

1. **Dissonanze Concettuali** : utenti e sviluppatori possono dare significati diversi agli stessi concetti, quindi è necessario definire in modo preciso i termini utilizzati.
I concetti utilizzati in una applicazione devono essere definiti con rigore riferendosi a glossari per verificare se ce discordanza di significati
2. **Struttura Complessa** : un Dominio Applicativo include un insieme di Processi di Business che hanno molte relazioni tra loro da dover essere gestite.
3. **Carenza di conformità** : La conformità esprime la corrispondenza delle attività eseguite, dei metodi, delle tecniche e dei strumenti utilizzati in un processo software con gli omologhi presenti nella sua definizione formale, in mancanza di questa si crea carenza di qualità

I PROCESSI SOFTWARE

Oggi i requisiti cambiano molto velocemente, quindi i processi sequenziali come il modello a cascata non sono sempre adatti. Esistono quindi due grandi approcci:

- tecniche basate sull'architettura: cercano di prevedere i cambiamenti e usano l'information hiding e la modularità per nascondere le sorgenti di cambiamento nei moduli dell'architettura
- tecniche basate sul refactoring: adattano il software nel tempo con modifiche incrementali rendendo il prodotto il più semplice possibile con cambiamenti dei requisiti

Scelta della classe più appropriata

- Se l'evoluzione dei requisiti si può conoscere per tempo e i contenuti restano stabili per un tempo sufficiente allora andiamo per Tecniche sull'Architettura.
- Se i cambiamenti sono frequenti e difficilmente prevedibili o del tutto imprevedibili allora andiamo per Tecniche sul Refactoring.

I modelli di processo

Le tecniche basate sull'architettura sono tipiche dei processi guidati da piani, come Waterfall, Spirale e Rational Unified Process. Le tecniche basate sul refactoring sono tipiche dei processi agili, come Scrum, Extreme Programming e Agile. In molti casi si usano approcci misti.